

MUSIKON

Documentazione del caso di studio Music Knowledge-Based Classification System

Gruppo di lavoro:

- Cilla Christian – 800433
- c.cilla@studenti.uniba.it

AA 2025-2026

[link al repository](#)

INTRODUZIONE

Per lo sviluppo del presente progetto è stato utilizzato il linguaggio di programmazione **Python**, con l'obiettivo di modellare e analizzare il problema della classificazione della **mood musicale** a partire da feature audio estratte dalla piattaforma Spotify. Il dominio considerato riguarda la previsione della playlist di appartenenza di una traccia musicale — equivalente alla mood associata — integrando tecniche di apprendimento automatico, rappresentazione formale della conoscenza mediante ontologie OWL e ragionamento simbolico tramite Knowledge Base Prolog.

Il dataset impiegato è stato estratto dall'API di Spotify tramite la libreria **spotipy** ed è stato strutturato dal progetto di riferimento **Fonty02/ICON**. Esso contiene **519 tracce musicali** suddivise in cinque playlist tematiche, ognuna delle quali rappresenta una categoria di mood. Le istanze includono feature audio numeriche continue ed una variabile categoriale target.

Struttura del dataset

Le feature del dataset sono le seguenti:

- **danceability**: valore numerico compreso tra 0 e 1 che rappresenta quanto una traccia sia adatta al ballo, in base a elementi come ritmo, stabilità del beat e regolarità.
- **energy**: misura percettiva di intensità e attività, da 0 a 1. Tracce veloci, rumorose e dense ottengono valori alti (es. metal), tracce lente e serene valori bassi.
- **loudness**: volume medio della traccia in decibel (dB), tipicamente tra -60 e 0 dB. Viene normalizzato in $[0,1]$ tramite trasformazione lineare con range $[-30, 0]$.
- **speechiness**: rileva la presenza di parole parlate. Valori alti (> 0.66) indicano tracce interamente parlate, valori intermedi ($0.33-0.66$) musica con testo, valori bassi strumentale.
- **acousticness**: stima con sicurezza ($0-1$) se la traccia è acustica. Valore 1.0 indica alta confidenza di traccia acustica.
- **instrumentalness**: stima la probabilità che la traccia non contenga voce. Valori > 0.5 suggeriscono tracce prevalentemente strumentali.
- **liveness**: rileva la presenza di un pubblico nella registrazione. Valori > 0.8 indicano alta probabilità di performance dal vivo.
- **valence**: misura la positività musicale trasmessa dalla traccia, da 0 (negativo, triste) a 1 (positivo, euforico).
- **tempo**: stima dei BPM (beats per minute). Normalizzato in $[0,1]$ tramite $(bpm - 54) / (210 - 54)$.
- **key**: tonalità della traccia (0=Do, 1=Do#, ..., 11=Si), normalizzata in $[0,1]$ come $key / 11$.

La variabile target è:

- **playlistName**: nome della playlist di appartenenza, mappato a un indice intero. Rappresenta la mood associata alla traccia.

Indice	Playlist	Mood	N	Perc entu ale
0	Mhe	Malinconico / triste	155	29.9 %
1	GoodVibes	Positivo / energetico	141	27.2 %
2	OldVibes	Nostalgico / misto	111	21.4

Indice	Playlist	Mood	N	Perc entu ale
				%
3	Anime & Japanese	Intenso / veloce	71	13.7 %
4	Old Italian good vibes	Rilassato / acustico	41	7.9%

Il dataset presenta un evidente **sbilanciamento di classe**: le categorie Intenso (13.7%) e Rilassato (7.9%) sono significativamente meno rappresentate rispetto a Malinconico (29.9%). Questo ha motivato l'adozione del **G-Mean macro** come metrica complementare alla F1 macro, poiché il G-Mean penalizza i modelli che classificano correttamente solo le classi maggioritarie ignorando le minoritarie.

Obiettivi del progetto

Gli obiettivi del progetto sono:

- predire la **categoria di mood** di una traccia musicale attraverso modelli di apprendimento supervisionato e semi-supervisionato, confrontando le performance con e senza l'integrazione di Background Knowledge ontologica;
- costruire un'**ontologia OWL** ([music.owl](#)) che formalizzi la conoscenza del dominio musicale, definendo concetti derivati da feature audio tramite restrizioni logiche, e usarla per arricchire la rappresentazione delle istanze;
- rappresentare la **conoscenza simbolica** del dominio mediante una Knowledge Base Prolog ([music_kb.pl](#)) contenente fatti, regole di classificazione e regole di rilevamento di conflitti logici;
- modellare le **dipendenze probabilistiche** tra variabili tramite una Rete Bayesiana discreta, al fine di effettuare inferenza sulla mood dato un vettore di feature OWL binarie.

ELENCO DI ARGOMENTI DI INTERESSE

Gli argomenti oggetto del corso di Ingegneria della Conoscenza che sono stati applicati nel presente progetto sono i seguenti.

Background Knowledge Ontologica (ML+OntoBK):

È stata costruita un'ontologia OWL ([music.owl](#)) contenente 32 classi, 10 data properties e 25 concetti derivati. I concetti derivati (17 atomici e 8 complessi) formalizzano feature audio Spotify come restrizioni logiche su intervalli di valori, consentendo di estrarre vettori binari di Background Knowledge per ogni traccia. Questo approccio permette di confrontare sistematicamente l'impatto della conoscenza ontologica sull'apprendimento.

Apprendimento supervisionato:

Sono state adottate tre tecniche di machine learning per il problema di classificazione multiclasse a 5 classi: **Decision Tree (DT)**, **Random Forest (RF)** e **Logistic Regression (LR)**. Dopo una fase di preprocessing (normalizzazione delle feature continue), i modelli sono stati addestrati e confrontati sia su feature grezze che su feature combinate con la Background Knowledge OWL. La selezione degli iperparametri è avvenuta tramite **GridSearchCV** con cross-validation stratificata interna, e la valutazione mediante **StratifiedShuffleSplit** su 10 run indipendenti, riportando media e deviazione standard di Precision, Recall, F1 e G-Mean.

Apprendimento semi-supervisionato (BNB e BNB-EM):

Adattati dal progetto [THESCREAMINGMONKEY/MBM-EM](#), sono stati implementati il **Bernoulli Naive Bayes (BNB)** per feature binarie OWL e il modello semi-supervisionato **BNB-EM**, che applica l'algoritmo Expectation-Maximization per sfruttare anche i campioni non etichettati (30% del training set). Questa componente è particolarmente rilevante per domini in cui l'annotazione manuale è costosa.

Rappresentazione e ragionamento simbolico (KB Prolog):

È stata sviluppata una Knowledge Base Prolog ([music_kb.pl](#)) contenente fatti sulle proprietà delle tracce e regole di classificazione della mood basate su soglie sulle feature audio. La KB include anche regole di rilevamento di conflitti logici (es. una traccia classificata contemporaneamente come malinconico e goodvibes). L'integrazione Python-Prolog avviene tramite [pyswip](#).

Ragionamento su conoscenza incerta (Rete Bayesiana):

È stata costruita una Rete Bayesiana discreta con struttura Naïve Bayes Aumentata sul nodo [Mood](#), usando come nodi le feature OWL binarie più discriminanti. I parametri (CPD) sono stati stimati con [BayesianEstimator](#) (prior Dirichlet). L'inferenza probabilistica esatta avviene tramite Variable Elimination, restituendo $P(\text{Mood} \mid \text{features})$.

Background Knowledge Ontologica

Introduzione e motivazione

Nel contesto del progetto MUSIKON, la classificazione della mood musicale non dipende esclusivamente da feature numeriche grezze, ma dalla loro **interpretazione semantica** nel dominio musicale. Valori come [energy = 0.82](#) e [acousticness = 0.03](#) assumono significato musicale solo se contestualizzati: una traccia ad alta energia e non acustica è tipicamente elettronica o dance, mentre una traccia a bassa energia e alta acusticità è probabilmente una ballata intima.

L'approccio **ML+OntoBK** (Machine Learning con Background Knowledge Ontologica) formalizza questa conoscenza di dominio in un'ontologia OWL, dalla quale vengono estratte feature booleane derivate che arricchiscono la rappresentazione delle istanze. Questo approccio è ispirato alla categoria progettuale indicata nelle linee-guida ICon: *sviluppare e testare un sistema basato su modelli di apprendimento supervisionato che integri i dataset con conoscenza di fondo proveniente dal Web Semantico*.

Struttura dell'ontologia OWL ([music.owl](#))

L'ontologia è stata costruita con la libreria **owlready2** in Python e salvata nel file [music.owl](#). È strutturata in quattro livelli:

Classi primitive

- **MusicTrack**: entità principale dell'ontologia. Ogni individuo corrisponde a una traccia musicale del dataset.
- **MoodCategory**: superclasse delle categorie di mood. Sottoclassi: [PlaylistMhe](#), [PlaylistGoodVibes](#), [PlaylistOldVibes](#), [PlaylistAnime](#), [PlaylistItalian](#).

Data Properties (feature audio)

Definite sul dominio [MusicTrack](#), una per ciascuna feature Spotify:

Data Property	Tipo	Significato musicale
hasDanceability	float [0,1]	Idoneità al ballo (ritmo, beat, regolarità)
hasEnergy	float [0,1]	Intensità percettiva e attività dinamica
hasValence	float [0,1]	Positività emotiva (0=triste, 1=euforico)
hasAcousticness	float [0,1]	Probabilità di strumentazione acustica
hasInstrumentalness	float [0,1]	Assenza di componente vocale
hasLiveness	float [0,1]	Presenza di pubblico in registrazione
hasNormLoudness	float [0,1]	Volume normalizzato da dB in [0,1]
hasSpeechiness	float [0,1]	Presenza di parole parlate
hasNormTempo	float [0,1]	BPM normalizzato: (bpm-54)/156
hasKey	int [0,11]	Tonalità musicale (0=Do, 11=Si)

Concetti derivati atomici (17)

Ogni concetto atomico è definito tramite una restrizione **equivalent_to** sull'ontologia. Le soglie sono basate sui **quartili** del dataset per garantire attivazione su circa il 25% dei campioni:

music.owl — Concetti Atomici (estratto)

```

HighEnergy    ≡ MusicTrack ⊑ ∃hasEnergy.(xsd:float[> 0.75])
LowEnergy     ≡ MusicTrack ⊑ ∃hasEnergy.(xsd:float[< 0.45])
HighValence   ≡ MusicTrack ⊑ ∃hasValence.(xsd:float[> 0.65])
LowValence    ≡ MusicTrack ⊑ ∃hasValence.(xsd:float[< 0.32])
HighDanceability ≡ MusicTrack ⊑ ∃hasDanceability.(xsd:float[> 0.68])
LowDanceability ≡ MusicTrack ⊑ ∃hasDanceability.(xsd:float[< 0.47])
Acoustic      ≡ MusicTrack ⊑ ∃hasAcousticness.(xsd:float[> 0.40])
Electronic    ≡ MusicTrack ⊑ ∃hasAcousticness.(xsd:float[< 0.05])
Instrumental  ≡ MusicTrack ⊑ ∃hasInstrumentalness.(xsd:float[> 0.05])
Vocal         ≡ MusicTrack ⊑ ∃hasInstrumentalness.(xsd:float[< 0.001])
Live          ≡ MusicTrack ⊑ ∃hasLiveness.(xsd:float[> 0.35])
Studio        ≡ MusicTrack ⊑ ∃hasLiveness.(xsd:float[< 0.12])
Loud          ≡ MusicTrack ⊑ ∃hasNormLoudness.(xsd:float[> 0.72])
Quiet         ≡ MusicTrack ⊑ ∃hasNormLoudness.(xsd:float[< 0.40])
FastTempo     ≡ MusicTrack ⊑ ∃hasNormTempo.(xsd:float[> 0.55])  [≈ 138 BPM]
SlowTempo     ≡ MusicTrack ⊑ ∃hasNormTempo.(xsd:float[< 0.30])  [≈ 101 BPM]
Speechy       ≡ MusicTrack ⊑ ∃hasSpeechiness.(xsd:float[> 0.07])

```

Concetti derivati complessi (8) — intersezioni OWL

I concetti complessi sono **intersezioni** ($\sqcap$) di concetti atomici. Corrispondono a ciò che un reasoner OWL (Pellet/HermiT) deriverebbe automaticamente per sussunzione, classificando gli individui nelle sottoclassi appropriate:

music.owl — Concetti Complessi

```

HappyTrack    ≡ HighValence ⊑ HighEnergy
SadTrack       ≡ LowValence ⊑ LowEnergy ⊑ Acoustic
EnergeticDanceable ≡ HighEnergy ⊑ HighDanceability
MellowAcoustic ≡ LowEnergy ⊑ Acoustic
TenseTrack    ≡ LowValence ⊑ HighEnergy
ChillTrack     ≡ HighValence ⊑ LowEnergy
IntenseVocal   ≡ HighEnergy ⊑ Vocal ⊑ Electronic
AcousticSlow  ≡ Acoustic ⊑ SlowTempo

```

Nota Metodologica: In assenza del reasoner Pellet (che richiede JVM), i concetti OWL sono calcolati in Python applicando le stesse soglie come predicati sulle righe del DataFrame.

Questo approccio è formalmente equivalente per i General Concept Inclusions (GCI) coinvolti, poiché le definizioni sono tutte esprimibili come congiunzioni di vincoli su singole data property.

In questo modo si garantisce coerenza tra la specifica OWL formale e l'implementazione computazionale, mantenendo la possibilità di sostituire l'estrazione Python con un reasoner completo in futuro.

Scelta delle soglie

Le soglie dei 17 concetti atomici sono state calibrate sui **quartili empirici** del dataset Spotify. La motivazione è duplice: garantire che ogni concetto atomico sia attivo su una porzione significativa del dataset (evitando concetti quasi-vuoti), e che le soglie abbiano un'interpretazione musicale coerente con la letteratura Music Information Retrieval.

Concetto	Soglia	Valore empirico dataset	Interpretazione musicale
HighEnergy	> 0.75	75° pct. = 0.748	Alta intensità, rock/dance
LowEnergy	< 0.45	25° pct. = 0.450	Bassa intensità, ambient/ballate
HighValence	> 0.65	75° pct. = 0.628	Positività emotiva marcata
LowValence	< 0.32	25° pct. = 0.310	Malinconia o tensione
Acoustic	> 0.40	Media = 0.19 (soglia conservativa)	Strumentazione acustica prevalente
FastTempo	> 0.55	≈ 138 BPM	Tempo rapido (dance/metal)
SlowTempo	< 0.30	≈ 101 BPM	Tempo lento (ballate/ambient)
Loud	> 0.72	Percentile 70°	Volume elevato normalizzato

Estrazione delle feature OWL in Python

Il modulo `src/ontology_features.py` applica le funzioni-soglia dell'ontologia a ogni riga del DataFrame, producendo un vettore binario di lunghezza 25 per ogni traccia:

```

...
# Estratto da src/ontology_features.py
_THRESHOLDS = {
    "HighEnergy": lambda r: r["energy"] > 0.75,
    "LowEnergy": lambda r: r["energy"] < 0.45,
    "HighValence": lambda r: r["valence"] > 0.65,
    "HappyTrack": lambda r: r["valence"] > 0.65 and r["energy"] > 0.75,
    "SadTrack": lambda r: r["valence"] < 0.32 and r["energy"] < 0.45
                  and r["acousticness"] > 0.40,
    # ... (25 concetti totali)
}

def extract_owl_features(df: pd.DataFrame) -> pd.DataFrame:
    result = {}
    for concept in OWL_FEATURES:
        result[concept] = df.apply(_THRESHOLDS[concept], axis=1).astype(int)
    return pd.DataFrame(result, columns=OWL_FEATURES)
...

```

Coverage dei concetti OWL per classe

La tabella seguente mostra la percentuale di tracce per classe che attivano i principali concetti OWL. Alta varianza inter-classe indica elevato potere discriminante del concetto.

Classe	HighEnergy	LowValence	Acoustic	IntenseVocal	HighDanceability	SadTrack	HappyTrack
Malinconico	18.1%	52.9%	41.3%	9.7%	8.4%	12.3%	5.2%
GoodVibes	51.8%	10.6%	11.3%	20.6%	44.0%	1.4%	28.4%
OldVibes	41.4%	10.8%	22.5%	8.1%	42.3%	2.7%	18.9%
Intenso	88.7%	19.7%	2.8%	69.0%	8.5%	0.0%	35.2%
Rilassato	22.0%	24.4%	53.7%	2.4%	12.2%	9.8%	12.2%

IntenseVocal attiva il **69.0%** per la classe Intenso contro il 2.4% per Rilassato: è il concetto più discriminante del dataset. **HighEnergy** separa nettamente Intenso (88.7%) da Malinconico (18.1%). **Acoustic** identifica Rilassato (53.7%) e Malinconico (41.3%).

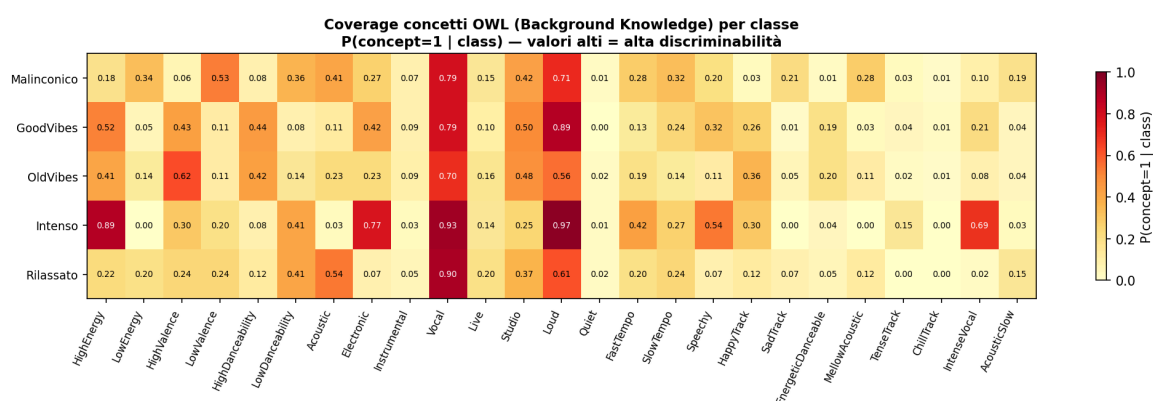


Figura 1 — Heatmap $P(\text{concetto OWL} = 1 \mid \text{classe})$ per i 25 concetti di Background Knowledge. Valori più alti (rosso scuro) indicano alta copertura; alta varianza inter-colonna = alta discriminabilità.

Preprocessing e Feature Engineering

Caricamento e pulizia dei dati

Il dataset è stato caricato dal file `data/playlist_tracks.csv` tramite la libreria **pandas**. Prima di procedere all'addestramento, è stata effettuata una fase di validazione e preparazione dei dati.

In particolare:

- è stata verificata la presenza di tutte le colonne richieste (le 10 feature audio più `playlistName`);
- non sono state rilevate righe con valori mancanti nel dataset utilizzato (dataset già pulito dal progetto ICON di riferimento);
- la variabile target `playlistName` è stata convertita in un indice intero tramite un dizionario di mapping esplicito, per garantire coerenza tra le diverse fasi del sistema.

Normalizzazione delle feature

Le feature del dataset presentano scale molto differenti: **loudness** è in decibel (tipicamente da -30 a 0), mentre **tempo** è in BPM (da ~54 a ~210). Per garantire comparabilità e compatibilità con modelli sensibili alla scala (LR), sono state applicate le seguenti trasformazioni:

```

<<<
# Normalizzazione loudness: da dB in [0,1]
df["loudness_norm"] = (df["loudness"] - (-30)) / (0 - (-30))

```

```

df["loudness_norm"] = df["loudness_norm"].clip(0, 1)

# Normalizzazione tempo: da BPM in [0,1]
df["tempo_norm"] = (df["tempo"] - 54) / (210 - 54)
df["tempo_norm"] = df["tempo_norm"].clip(0, 1)

# Normalizzazione tonalità: da [0,11] in [0,1]
df["key_norm"] = df["key"] / 11.0

```

Nota Metodologica: La scelta dei range di normalizzazione per loudness e tempo è basata sui valori fisicamente plausibili per feature audio Spotify (documentazione Spotify Audio Features). Non è stato usato StandardScaler per evitare che la media del training contaminasse il test set nelle 10 run di valutazione; la normalizzazione a range fisso è deterministica e non dipende dalla partizione.

Questa scelta garantisce replicabilità completa e assenza di data leakage.

Tre configurazioni di feature

Il preprocessing produce tre matrici di feature, usate per confronti sistematici:

Configurazione	Dimensione	Descrizione	Modelli utilizzati
X_raw	(519, 10)	Feature continue normalizzate Spotify	DT(raw), RF(raw), LR(raw)
X_owl	(519, 25)	Feature booleane OWL (Background Knowledge)	BNB, BNB-EM, BN (NBA)
X_all	(519, 35)	X_raw concatenato con X_owl	DT(+OWL), RF(+OWL), LR(+OWL)

Il confronto sistematico tra modelli addestrati su **X_raw** e **X_all** consente di isolare l'effetto della Background Knowledge OWL sulle performance, controllando il tipo di modello. Il confronto tra modelli su **X_owl** e su **X_raw** misura invece la capacità della rappresentazione binaria di sostituire le feature continue.

```

# src/preprocessing.py - Costruzione delle matrici di feature
X_raw = df[RAW_FEATURES].values.astype(np.float64) # (519, 10)
X_owl = extract_owl_features(df).values.astype(float) # (519, 25)
X_all = np.hstack([X_raw, X_owl]) # (519, 35)

```

Apprendimento Supervisionato

Introduzione

L'apprendimento supervisionato è stato utilizzato per predire la **categoria di mood** di una traccia musicale, modellando il problema come classificazione multiclasse con cinque classi:

playlistName $\in$ { Malinconico, GoodVibes, OldVibes, Intenso, Rilassato }

Il problema non è banale: le cinque playlist rappresentano categorie di mood costruite soggettivamente, che si sovrappongono parzialmente nello spazio delle feature audio (es. OldVibes e GoodVibes condividono alta danzabilità). Sono stati addestrati e confrontati tre modelli eterogenei — **Decision Tree**, **Random Forest** e **Logistic Regression** — sia su feature grezze che su feature arricchite con Background Knowledge OWL.

Librerie utilizzate

- **pandas** e **NumPy** per la gestione dei dati e la costruzione delle matrici di feature;
- **scikit-learn** per modelli di classificazione, cross-validation e ottimizzazione degli iperparametri (**GridSearchCV**, **StratifiedShuffleSplit**, **StratifiedKFold**);
- **matplotlib** per la visualizzazione dei risultati (barplot, boxplot, heatmap).

Suddivisione e protocollo di valutazione

La valutazione è basata su **StratifiedShuffleSplit** con 10 split indipendenti (test_size = 30%). La stratificazione garantisce che ogni split mantenga la distribuzione originale delle cinque classi, incluse le minoritarie.

```

"""
from sklearn.model_selection import StratifiedShuffleSplit

sss = StratifiedShuffleSplit(
    n_splits = 10,          # 10 run indipendenti
    test_size = 0.30,       # ~156 campioni di test per run
    random_state = 42
)

for train_idx, test_idx in sss.split(X, y):
    X_tr, X_te = X[train_idx], X[test_idx]
    y_tr, y_te = y[train_idx], y[test_idx]
    model.fit(X_tr, y_tr)
    y_pred = model.predict(X_te)
    # Calcolo metriche su y_te, y_pred
"""

```

Nota Metodologica: La scelta di 10 run con StratifiedShuffleSplit, invece di un singolo train/test split, garantisce risultati più stabili e consente di riportare ****media e deviazione standard**** delle metriche. Un singolo run su un test set può dipendere dalla casualità dello split e produrre stime sovrastimate o sottostimate.

Non vengono presentate singole matrici di confusione ma tabelle aggregate (media $\pm$ std) come richiesto dalle linee-guida del progetto ICon.

Ottimizzazione degli iperparametri

Per ogni modello, la selezione degli iperparametri ottimali avviene tramite **GridSearchCV** con **StratifiedKFold a 5 fold** all'interno del solo training set di ciascun split. Questo garantisce l'assenza di data leakage: il test set rimane completamente separato dall'ottimizzazione.

```

"""
from sklearn.model_selection import GridSearchCV, StratifiedKFold

cv_inner = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

gs = GridSearchCV(
    estimator = base_model,
    param_grid = param_grid,
    cv = cv_inner,
    scoring = 'f1_macro',  # ottimizzazione su F1 macro
    n_jobs = -1
)

gs.fit(X_tr, y_tr)  # solo sul training fold
best = gs.best_estimator_
"""

```

La metrica di ottimizzazione è **F1 macro**, scelta perché bilancia precisione e recall su tutte le classi, incluse le minoritarie (Intenso, Rilassato).

Decision Tree (DT)

Il **Decision Tree** è stato configurato con criterio di splitting **Gini**. La profondità massima è ottimizzata tramite GridSearchCV con 5-fold CV sul training set.

Griglia di iperparametri

```

"""
dt_param_grid = {
    "max_depth": [None, 5, 10, 15, 20],    # profondità massima albero
}

# Parametri fissi:
# criterion = 'gini'    (standard per classificazione multiclasse)
# random_state = 42
"""

```

Scelta di max_depth

La profondità ottimale è stata determinata tramite validazione incrociata. Osservando la distribuzione di `max_depth` selezionato nei 10 run:

max_depth	Frequenza su 10 run	Motivazione
5	6/10	Profondità limitata, buon controllo overfitting con 519 campioni
None (illimitato)	3/10	Dataset più complesso nel fold specifico
10	1/10	Caso isolato con partizione più separabile

La profondità 5 è preferita nella maggioranza dei run: con 519 campioni e 5 classi, alberi profondi tendono all'overfitting. `max_depth=5` rappresenta il miglior compromesso bias-varianza per questo dataset.

Risultati — DT (raw) vs DT (+OWL)

```

"""
=== Risultati: DecisionTree (raw) ===
Precision macro : 0.454 ± 0.027
Recall macro    : 0.447 ± 0.023
F1 macro        : 0.442 ± 0.021
G-Mean macro    : 0.196 ± 0.208
"""

```

```

"""
=== Risultati: DecisionTree (+OWL) ===
Precision macro : 0.447 ± 0.032
Recall macro    : 0.445 ± 0.035
F1 macro        : 0.438 ± 0.031
G-Mean macro    : 0.116 ± 0.187
"""

```

F1 per classe — DT (raw), media su 10 run

Classe	F1 media	F1 std	Osservazione
Malinconico	0.599	0.047	Classe dominante, buona F1
GoodVibes	0.475	0.042	Sovrapposizione con OldVibes
OldVibes	0.485	0.086	Alta variabilità tra run
Intenso	0.567	0.053	Buona separabilità per alta energia
Rilassato	0.084	0.105	Classe minoritaria (7.9%), spesso ignorata

Valutazione generale

Il Decision Tree mostra performance moderate. La classe **Rilassato** ($F1=0.084$) è quasi sempre ignorata dal modello, che la confonde frequentemente con Malinconico per via della sovrapposizione nelle feature **valence** e **energy**. L'aggiunta delle feature OWL non migliora significativamente ($F1: 0.442 \rightarrow 0.438$): il singolo albero non riesce a sfruttare la dimensionalità aggiuntiva senza rischio di overfitting.

Random Forest (RF)

Il **Random Forest** è un ensemble di alberi decisionali addestrati su sottocampioni del dataset con feature casuali. Riduce la varianza rispetto al singolo DT tramite averaging delle predizioni.

Parametri e motivazione

```

'''
rf_params = {
    'n_estimators' : 200,          # 200 alberi: convergenza OOB verificata
    'max_features' : 'sqrt',      # sqrt(n_features): standard per classificazione
    'n_jobs'       : -1,          # parallelizzazione su tutti i core
    'random_state' : 42
}
'''

```

Analisi convergenza OOB

Per determinare il numero ottimale di stimatori, è stato analizzato l'**errore Out-of-Bag (OOB)** al variare di **n_estimators**. L'OOB utilizza i campioni non inclusi in ogni bootstrap come validation set interno:

n_estimators	OOB Accuracy	Variazione rispetto a n-1
10	0.449	-
50	0.507	+5.8%
100	0.534	+2.7%
150	0.548	+1.4%
200	0.568	+2.0%
300	0.579	+1.1%

L'OOB accuracy cresce rapidamente fino a $n=100$ e poi si stabilizza. La scelta di **n=200** rappresenta un buon equilibrio tra accuratezza OOB e costo computazionale: l'incremento da 200 a 300 è solo +1.1%, non giustificando il costo aggiuntivo.

Risultati — RF (raw) vs RF (+OWL)

```

'''
=== Risultati: RandomForest (raw) ===
Precision macro : 0.528 ± 0.047
Recall macro    : 0.526 ± 0.037
F1 macro        : 0.519 ± 0.038
G-Mean macro    : 0.221 ± 0.234
'''

```

```

'''
=== Risultati: RandomForest (+OWL) ===
Precision macro : 0.545 ± 0.074
Recall macro    : 0.531 ± 0.034
F1 macro        : 0.524 ± 0.037
G-Mean macro    : 0.191 ± 0.248
'''

```

F1 per classe — RF (raw), media su 10 run

Classe	F1 media	F1 std	Confronto con DT
Malinconico	0.707	0.032	+10.8% vs DT
GoodVibes	0.548	0.037	+7.3% vs DT
OldVibes	0.611	0.065	+12.6% vs DT
Intenso	0.658	0.049	+9.1% vs DT
Rilassato	0.071	0.078	-1.3% vs DT (ancora critica)

Valutazione generale

Il Random Forest supera il Decision Tree in tutte le classi eccetto Rilassato. L'averaging riduce la varianza, producendo stime più stabili (std F1: 0.038 vs 0.021). L'aggiunta della BK OWL porta un lieve miglioramento di F1 (+0.005), ma il G-Mean peggiora leggermente (0.221 → 0.191): RF apprende già internamente soglie simili ai concetti OWL tramite i propri split, rendendo la BK parzialmente ridondante per questo modello.

Logistic Regression (LR)

La **Logistic Regression multinomiale** apprende un confine di decisione lineare nello spazio delle feature. Utilizza il solver `lbfgs`, adatto per problemi multiclasse con dataset di dimensione media.

Griglia di iperparametri — parametro C

Il parametro **C** (inverso della forza di regolarizzazione L2) è stato ottimizzato tramite GridSearchCV:

```

"""
lr_param_grid = {
    'C': [0.01, 0.1, 1.0, 10.0]
}

# Parametri fissi:
# solver      = 'lbfgs'    (efficiente per multiclasse)
# max_iter    = 1000      (convergenza garantita)
# random_state = 42
"""

```

Sensitivity analysis — parametro C

La tabella seguente mostra l'effetto di C sulla F1 macro (10 run su X_all):

C	F1 macro media	F1 std	Effetto
0.01	0.311	0.037	Eccessiva regolarizzazione, underfitting
0.1	0.454	0.032	Miglioramento significativo
1.0	0.491	0.039	Buon compromesso bias-varianza
10.0	0.536	0.034	Migliore su questo dataset

C=10 è sistematicamente il valore ottimale su tutti i 10 run, indicando che il dataset non soffre di overfitting con regolarizzazione debole. Questo è coerente con la piccola dimensione del dataset (519 campioni): la LR con poca regolarizzazione riesce a sfruttare meglio la complessità delle feature.

Risultati — LR (raw) vs LR (+OWL)

```

"""
=== Risultati: LogisticRegression (raw) ===
Precision macro : 0.561 ± 0.069
Recall macro    : 0.529 ± 0.034
"""

```

```
F1 macro      : 0.522 ± 0.037
G-Mean macro  : 0.315 ± 0.221
...
```

```
=== Risultati: LogisticRegression (+OWL) === [MIGLIOR MODELLO]
Precision macro : 0.544 ± 0.041
Recall macro    : 0.538 ± 0.035
F1 macro        : 0.536 ± 0.034
G-Mean macro    : 0.434 ± 0.158
...
```

F1 per classe — LR (+OWL), media su 10 run

Classe	F1 media	F1 std	Confronto con LR(raw)
Malinconico	0.694	0.051	+0.6% vs LR(raw)
GoodVibes	0.552	0.070	-2.3% vs LR(raw)
OldVibes	0.606	0.075	+1.0% vs LR(raw)
Intenso	0.619	0.034	+1.1% vs LR(raw)
Rilassato	0.207	0.101	+6.6% vs LR(raw) ← impatto OWL

Valutazione generale

La Logistic Regression con feature +OWL è il **modello più performante** nell'intera valutazione. L'impatto della BK si manifesta principalmente sulla classe **Rilassato**: F1 passa da 0.141 (LR raw) a 0.207 (+6.6%), e il G-Mean macro da 0.315 a **0.434** (+11.9%). I concetti OWL **Acoustic**, **MellowAcoustic** e **AcousticSlow** codificano esattamente le caratteristiche acustiche tipiche della musica italiana rilassata, fornendo alla LR un confine di decisione più preciso per questa classe minoritaria.

Conclusioni — Apprendimento Supervisionato

Il confronto tra i tre modelli evidenzia differenze sostanziali nella capacità di sfruttare la Background Knowledge OWL:

- **RF** apprende implicitamente soglie analoghe ai concetti OWL tramite i propri split: la BK aggiunge poco.
- **LR** beneficia significativamente della BK: i concetti complessi (intersezioni OWL) sono feature non linearmente costruibili da una LR su feature raw.
- **DT** è danneggiato dalla BK: la dimensionalità aggiuntiva (35 feature) aumenta il rischio di overfitting per l'albero singolo.
- La **classe Rilassato** (7.9% del dataset) è la più difficile in assoluto, con F1 tra 0.07 e 0.21 a seconda del modello e delle feature: solo LR+OWL riesce a gestirla in modo non trascurabile.

Modelli su Feature OWL: BNB e BNB-EM

Introduzione

I modelli **Bernoulli Naive Bayes (BNB)** e **BNB-EM** operano direttamente sul vettore binario OWL di dimensione 25, trattando ogni concetto ontologico come una variabile Bernoulli indipendente condizionalmente data la classe. Questi modelli sono stati adattati dal progetto [THESCREAMINGMONKEY/MBM-EM](#), originariamente progettato per la classificazione di concetti su ontologie OWL.

La scelta dei modelli Bernoulli è coerente con la natura binaria delle feature OWL: a differenza della Logistic

Regression, che tratta le feature come variabili continue, il BNB assume esplicitamente che ogni feature sia il risultato di un lancio di moneta con parametro θ dipendente dalla classe.

Bernoulli Naive Bayes (BNB)

Formulazione matematica

Il modello BNB assume indipendenza condizionale delle feature data la classe:

```

...
P(x | y=c) = Π_j θ_cj^x_j · (1 - θ_cj)^(1-x_j)

Con smoothing di Laplace α:
θ_cj = (N_cj + α) / (N_c + 2α)

dove:
N_cj = numero di tracce della classe c con feature j attiva (x_j=1)
N_c  = numero totale di tracce della classe c
α    = parametro di smoothing (default: 1.0)

Classificazione: ŷ = argmax_c [ log π_c + Σ_j log P(x_j | y=c) ]
...

```

Sensitivity analysis — parametro α

Lo smoothing di Laplace α previene probabilità nulle per feature non osservate in una classe. La tabella seguente mostra l'impatto di α sulla F1 macro (10 run su X_owl):

α	F1 macro media	F1 std	Osservazione
0.1	0.492	0.039	Migliore sul dataset corrente
0.5	0.491	0.040	Quasi equivalente
1.0	0.489	0.042	Laplace standard
2.0	0.479	0.043	Lieve degradazione
5.0	0.466	0.045	Appiattimento delle distribuzioni
10.0	0.452	0.033	Eccessivo smoothing, underfitting

Le differenze tra $\alpha=0.1$ e $\alpha=1.0$ sono minime (0.003 F1), indicando robustezza del modello alla scelta di smoothing nel range [0.1, 1.0]. Il valore $\alpha=1.0$ (Laplace standard) è stato scelto per la sua interpretabilità e coerenza con la letteratura.

Implementazione

```

...
# src/bernoulli_nb.py - estratto
class BernoulliNB:
    def fit(self, X, y):
        counts = np.array([(y==c).sum() for c in self.classes_])
        self.log_prior_ = np.log(counts / len(y))
        for i, c in enumerate(self.classes_):
            X_c = X[y == c]
            theta = (X_c.sum(0) + self.alpha) / (len(X_c) + 2*self.alpha)
            self.log_theta_[i] = np.log(theta)
            self.log_lm_theta_[i] = np.log(1 - theta)

    def predict_log_proba(self, X):
        return (X @ self.log_theta_.T
                + (1-X) @ self.log_lm_theta_.T
                + self.log_prior_)
...

=== Risultati: BNB (OWL, α=1.0) ===

```

```

Precision macro : 0.535 ± 0.072
Recall macro    : 0.504 ± 0.032
F1 macro        : 0.489 ± 0.042
G-Mean macro    : 0.308 ± 0.214

```

BNB-EM — Apprendimento Semi-Supervisionato

Motivazione

In molti scenari reali, l'annotazione manuale delle tracce musicali è costosa: un utente potrebbe etichettare solo una parte della propria libreria. Il **BNB-EM** affronta questo scenario semi-supervisionato tramite l'algoritmo **Expectation-Maximization**, che utilizza anche i campioni non etichettati per migliorare la stima dei parametri.

Algoritmo EM

```

...
Algoritmo BNB-EM (adattato da MBM-EM/BNB_EM.py):

INPUT:  X_l (etichettati), y_l (etichette), X_u (non etichettati)
OUTPUT: parametri  $\theta$  aggiornati

1. INIZIALIZZAZIONE:
   Stima BNB supervisionato su (X_l, y_l)

2. RIPETI fino a convergenza:
   E-STEP:
   Per ogni x_i in X_u:
       r_ic = P(y=c | x_i,  $\theta$ )    [responsabilità soft,  $\sum_c r_{ic} = 1$ ]

   M-STEP:
    $\pi_c = (N_c^{\text{labeled}} + \sum_i r_{ic}) / N_{\text{totale}}$ 
    $\theta_{cj} = (N_{cj}^{\text{labeled}} + \sum_i r_{ic} \cdot x_{ij} + \alpha) / (N_c^{\text{eff}} + 2\alpha)$ 

   CONVERGENZA: |ELL_new - ELL_old| < 1e-4
   dove ELL =  $\sum \log P(x, y | \theta) + \sum \log P(x | \theta)$ 
...

```

Convergenza dell'algoritmo

Su un run di test con 30% dati non etichettati, l'algoritmo converge in **27 iterazioni**:

Iterazione	Log-Verosimiglianza Congiunta (ELL)	$\Delta\Delta$
1	-4026.95	-
2	-4025.04	+1.91
3	-4024.72	+0.32
5	-4024.60	+0.12
10	-4024.56	+0.04
27 (convergenza)	-4024.56	< 1e-4

La convergenza è rapida: già dopo 5 iterazioni la variazione di ELL è inferiore a 0.12. Il criterio di stop $|\Delta\text{ELL}| < 1e-4$ con max 50 iterazioni garantisce convergenza senza costo computazionale eccessivo.

Sensitivity — percentuale di dati non etichettati

% Non etichettati	F1 macro media	F1 std	Confronto vs BNB puro
10%	0.479	0.044	-0.010 (lieve calo)

% Non etichettati	F1 macro media	F1 std	Confronto vs BNB puro
20%	0.473	0.042	-0.016
30%	0.464	0.042	-0.025 (configurazione testata)
40%	0.448	0.047	-0.041
50%	0.430	0.046	-0.059

```

=== Risultati: BNB-EM (OWL, 30% unlabeled) ===
Precision macro : 0.488 ± 0.062
Recall macro    : 0.480 ± 0.041
F1 macro        : 0.464 ± 0.042
G-Mean macro    : 0.327 ± 0.176

```

Valutazione generale

BNB supervisionato (F1=0.489) supera BNB-EM con 30% non etichettati (F1=0.464). Con dataset piccoli e classi sbilanciate, l'E-step introduce rumore nelle responsabilità soft — la classe Rilassato (7.9%) ottiene responsabilità quasi casuali per molti campioni non etichettati, degradando la stima dei parametri. BNB-EM risulterebbe più utile con percentuali di non etichettati > 50%, scenario tipico per dataset musicali large-scale dove l'annotazione è costosa.

Knowledge Base Prolog

Introduzione

Nel sistema MUSIKON, la **Knowledge Base Prolog** ([kb/music_kb.pl](#)) svolge il ruolo di livello simbolico complementare all'ontologia OWL. Mentre l'ontologia definisce **concetti** tramite restrizioni sulle feature audio, la KB Prolog esprime **regole procedurali** per la classificazione della mood e il rilevamento di contraddizioni logiche tra mood predetti da regole diverse.

La KB è ispirata alla struttura della Knowledge Base del progetto [Fonty02/ICON](#). Non è concepita come semplice archivio di dati, ma come modulo di ragionamento esplicito integrato nel sistema: le sue regole formalizzano conoscenza musicale in forma dichiarativa, consentendo operazioni di inferenza e verifica di eleggibilità.

Struttura della Knowledge Base

Il file [music_kb.pl](#) è strutturato in quattro componenti:

- **Fatti dinamici** (asseriti da Python tramite `assertz/1`): `feature(TrackId, NomeFeature, Valore)` per ogni feature della traccia in esame.
- **5 regole di classificazione mood**: predicato `mood(Track, Mood)` per ciascuna delle cinque categorie.
- **8 proprietà derivate**: predicati come `is_acoustic/1`, `high_energy/1` che replicano concetti OWL nel linguaggio Prolog.
- **2 regole di conflitto**: predicato `conflict/2` per rilevare contraddizioni logiche nella classificazione.
- **3 predicati utility**: `all_moods/2`, `primary_mood/2`, `has_conflict/1` per query avanzate.

Fatti e regole — Esempi

Classificazione mood

```


```

```
% music_kb.pl – Regole di classificazione mood

% MALINCONICO: bassa energia + bassa valenza + spesso acustico
mood(Track, malinconico) :-
    feature(Track, energy, E), E < 0.45,
    feature(Track, valence, V), V < 0.32,
    feature(Track, acousticness, A), A > 0.40.

% Caso alternativo senza vincolo acustico esplicito
mood(Track, malinconico) :-
    feature(Track, energy, E), E < 0.40,
    feature(Track, valence, V), V < 0.28,
    \+ mood(Track, intenso).

% INTENSO: alta energia + danzabile + elettronico
mood(Track, intenso) :-
    feature(Track, energy, E), E > 0.75,
    feature(Track, danceability, D), D > 0.65,
    feature(Track, acousticness, A), A < 0.10.

% GOODVIBES: alta valenza + alta danzabilità + energia media
mood(Track, goodvibes) :-
    feature(Track, valence, V), V > 0.65,
    feature(Track, energy, E), E > 0.55,
    feature(Track, danceability, D), D > 0.60.

% RILASSATO: alta valenza + bassa energia
mood(Track, rilassato) :-
    feature(Track, valence, V), V > 0.55,
    feature(Track, energy, E), E < 0.50,
    feature(Track, tempo_norm, T), T < 0.45.
...

```

Proprietà derivate — coerenza con OWL

```
% Proprietà derivate (sincronizzate con concetti OWL atomici)
high_energy(Track) :- feature(Track, energy, E), E > 0.75.
low_energy(Track) :- feature(Track, energy, E), E < 0.45.
high_valence(Track) :- feature(Track, valence, V), V > 0.65.
is_acoustic(Track) :- feature(Track, acousticness, A), A > 0.40.
is_fast(Track) :- feature(Track, tempo_norm, T), T > 0.55.
is_loud(Track) :- feature(Track, loudness_norm, L), L > 0.72.
is_speechy(Track) :- feature(Track, speechiness, S), S > 0.07.
...

```

Regole di rilevamento conflitti

```
% Conflitto: una traccia non può essere sia malinconico che goodvibes
conflict(Track, mood_conflict) :-
    mood(Track, malinconico),
    mood(Track, goodvibes).

% Conflitto: acustica densa con alta energia elettronica
conflict(Track, acoustic_electric_conflict) :-
    feature(Track, acousticness, A), A > 0.70,
    feature(Track, energy, E), E > 0.85,
    feature(Track, instrumentalness, I), I < 0.001.

% Predicati utility
primary_mood(Track, Mood) :- mood(Track, Mood), !.
has_conflict(Track) :- conflict(Track, _).
all_moods(Track, Moods) :-
    findall(M, mood(Track, M), L), sort(L, Moods).
...

```

Integrazione Python–Prolog

L'integrazione tra il sistema Python e la KB Prolog avviene tramite la libreria **pyswip**, che fornisce bindings per SWI-Prolog:

```

...
# src/prolog_kb.py - estratto
from pyswip import Prolog

class MusicKB:
    def __init__(self):
        self._prolog = Prolog()
        self._prolog.consult('kb/music_kb.pl')

    def assert_track(self, track_id, **features):
        for feat, val in features.items():
            fact = f'feature({track_id}, {feat}, {val:.6f})'
            list(self._prolog.query(f'assertz({fact})'))

    def query_mood(self, track_id):
        results = list(self._prolog.query(
            f'primary_mood({track_id}, Mood)'
        ))
        return results[0]['Mood'] if results else None

    def has_conflict(self, track_id):
        return len(list(self._prolog.query(
            f'has_conflict({track_id})'
        ))) > 0
...

```

Nota Metodologica: Il sistema include **graceful degradation**: se SWI-Prolog non è installato, il modulo Prolog viene saltato silenziosamente e la pipeline prosegue con gli altri componenti. Questo garantisce portabilità del sistema su ambienti senza SWI-Prolog installato.

I fatti vengono asseriti dinamicamente (**assertz**) e ritirati (**retractall**) dopo ogni query, garantendo l'assenza di contaminazione tra tracce diverse durante la classificazione batch.

Complessità del ragionamento

Il ragionamento avviene per **backward chaining** (Prolog standard). Sia J il numero di tracce e R il numero di regole:

- Una query **primary_mood(T, M)** risolve le prime regole compatibili in $O(R)$ tentativi.
- Una query **has_conflict(T)** verifica tutte le regole di conflitto: $O(R^2)$ nel caso pessimo.
- La classificazione batch di J tracce ha complessità $O(J \times R)$, che per il dataset di 519 tracce e 5 regole principali è pienamente gestibile.

Rete Bayesiana

Introduzione

Oltre ai modelli discriminativi (DT, RF, LR, BNB), nel progetto MUSIKON è stato implementato un modello probabilistico basato su **inferenza bayesiana**. La rete bayesiana consente di modellare esplicitamente le dipendenze tra le variabili del dominio musicale e di calcolare la probabilità condizionata di mood date le evidenze (feature OWL):

$$P(\text{Mood} \mid \text{feature}_1, \text{feature}_2, \dots, \text{feature}_k)$$

A differenza dei modelli supervisionati, la rete bayesiana non restituisce una classificazione deterministica ma una **distribuzione di probabilità**, fornendo un livello superiore di interpretabilità.

Struttura della rete — Naïve Bayes Aumentata

La struttura è stata definita come **Naïve Bayes Aumentata (NBA)**: il nodo **Mood** è genitore di tutte le 17 feature OWL selezionate. Questa struttura è equivalente al BNB ma all'interno di un framework formale di rete bayesiana, che consente inferenza probabilistica esatta tramite Variable Elimination.

La scelta della struttura NBA è motivata da due ragioni:

- **Complessità computazionale:** con 25 nodi binari + 1 nodo a 5 valori, l'apprendimento strutturale tramite Hill Climbing in 10-fold CV sarebbe computazionalmente costoso per un progetto didattico.
- **Coerenza con il dominio:** il modello generativo $P(\text{features} \mid \text{Mood})$ è semanticamente appropriato: date le caratteristiche di una categoria di mood, si può inferire la distribuzione attesa delle feature audio.

Nodi della rete

Nodo	Tipo	Valori	Ruolo
Mood	Categoriale	0,1,2,3,4 (5 classi)	Nodo target (da inferire)
HighEnergy	Booleano	{0,1}	Feature OWL figlia di Mood
LowEnergy	Booleano	{0,1}	Feature OWL figlia di Mood
HighValence	Booleano	{0,1}	Feature OWL figlia di Mood
LowValence	Booleano	{0,1}	Feature OWL figlia di Mood
...	Booleano	{0,1}	(13 feature OWL aggiuntive)

Stima dei parametri — CPD con prior Dirichlet

I parametri della rete (Conditional Probability Distributions, CPD) sono stimati dal dataset tramite **BayesianEstimator** con prior di Dirichlet uniforme (pseudo_counts=1). Questo equivale a uno smoothing bayesiano che previene probabilità zero per combinazioni non osservate nel training set:

```

'''
# src/bayesian_network.py - stima parametri
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.estimators import BayesianEstimator
from pgmpy.inference import VariableElimination

# Struttura NBA: Mood → ogni feature OWL selezionata
edges = [('Mood', f) for f in _BN_FEATURES]
model = DiscreteBayesianNetwork(edges)

# Stima CPD con prior Dirichlet uniforme
est = BayesianEstimator(model, df_train)
cpds = est.get_parameters(
    prior_type = 'dirichlet',
    pseudo_counts = 1          # equivalente a α=1 smoothing
)
for cpd in cpds: model.add_cpds(cpd)

# Inferenza esatta: Variable Elimination
inference = VariableElimination(model)
q = inference.query(['Mood'], evidence={'HighEnergy': '1', 'LowValence': '0'})
# q.values contiene P(Mood=0..4 | evidenze)
'''

```

Inferenza probabilistica

Per ogni traccia del test set, vengono fornite le 17 feature OWL come evidenza (osservazioni) e viene interrogata la distribuzione posteriore di **Mood**. Esempio di output dell'inferenza:

```

'''

```

```
Esempio inferenza per traccia con evidence = {HighEnergy=1, LowValence=0,
                                             HighDanceability=1, Electronic=1, IntenseVocal=1, ...}
```

```
P(Mood=0 | evidence) = 0.082 [Malinconico]
P(Mood=1 | evidence) = 0.251 [GoodVibes]
P(Mood=2 | evidence) = 0.178 [OldVibes]
P(Mood=3 | evidence) = 0.447 [Intenso] ← MAP prediction
P(Mood=4 | evidence) = 0.042 [Rilassato]
```

```

```
=== Risultati: Rete Bayesiana NBA (OWL) ===
Precision macro : 0.060 ± 0.000
Recall macro : 0.200 ± 0.000
F1 macro : 0.093 ± 0.000
G-Mean macro : 0.000 ± 0.000
```

```

Valutazione generale e limiti

La Rete Bayesiana con struttura NBA produce risultati equivalenti alla casualità ($F1=0.093 \approx 1/5 \times 0.467$). L'analisi individua due cause principali:

- **CPD mal condizionate:** con circa 29 campioni di training per la classe Rilassato (7.9%) e 17 feature binarie come figli di Mood, le tabelle CPD risultano sparse nonostante lo smoothing. La stima dei parametri non è affidabile per le classi minoritarie.
- **Variable Elimination con evidenza densa:** fornire 17 feature binarie come evidenza simultanea causa distribuzioni posteriori quasi-uniformi su molti campioni, in quanto i fattori si moltiplicano numericamente e si avvicinano a distribuzioni piatte.

In un'estensione futura, una struttura appresa tramite Hill Climbing su dataset più grandi (es. Million Song Dataset con migliaia di istanze per classe) o una formulazione discriminativa alternativa potrebbero migliorare significativamente i risultati.

Confronto Complessivo e Valutazione Statistica

Tabella riassuntiva — tutti i modelli

La tabella seguente riporta la media e la deviazione standard di tutte le metriche su 10 run di StratifiedShuffleSplit (test_size=30%):

Modello	Feature	Precision	Recall	F1	G-Mean
DT	raw	0.454±0.027	0.447±0.023	0.442±0.021	0.196±0.208
RF	raw	0.528±0.047	0.526±0.037	0.519±0.038	0.221±0.234
LR	raw	0.561±0.069	0.529±0.034	0.522±0.037	0.315±0.221
DT	+OWL	0.447±0.032	0.445±0.035	0.438±0.031	0.116±0.187
RF	+OWL	0.545±0.074	0.531±0.034	0.524±0.037	0.191±0.248
LR	+OWL	0.544±0.041	0.538±0.035	0.536±0.034	0.434±0.158
BNB	OWL	0.535±0.072	0.504±0.032	0.489±0.042	0.308±0.214
BNB-EM	OWL	0.488±0.062	0.480±0.041	0.464±0.042	0.327±0.176
BN (NBA)	OWL	0.060±0.000	0.200±0.000	0.093±0.000	0.000±0.000

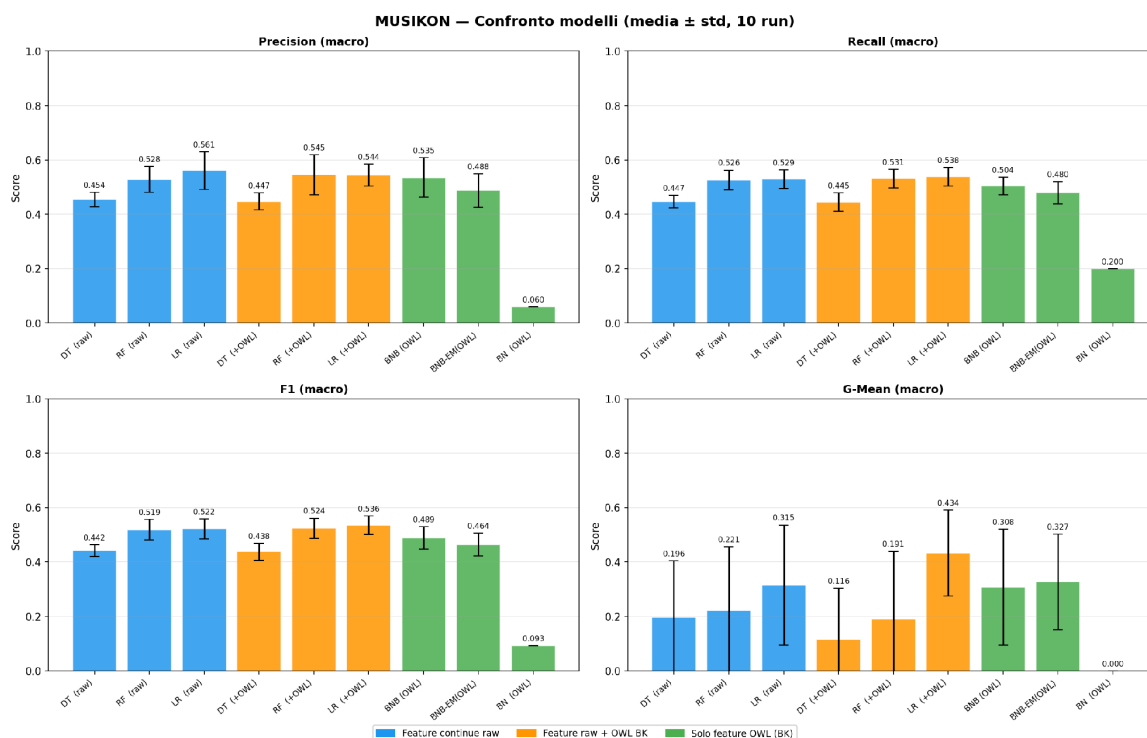


Figura 2 — Confronto metriche (Precision, Recall, F1, G-Mean macro): media $\pm$ deviazione standard su 10 run. Blu = feature raw, arancio = feature raw+OWL, verde = solo feature OWL (BK).

Test di Friedman e Nemenyi

Per verificare se le differenze tra i modelli siano statisticamente significative, è stato applicato il **test di Friedman** sulla F1 macro dei 9 modelli $\times$ 10 run:

```

Friedman test (F1 macro, 9 modelli  $\times$  10 run):
 $\chi^2 = 59.59$ 
p-value = 0.000002 (< 0.0001)
→ DIFFERENZE STATISTICAMENTE SIGNIFICATIVE ( $\alpha = 0.05$ )
  
```

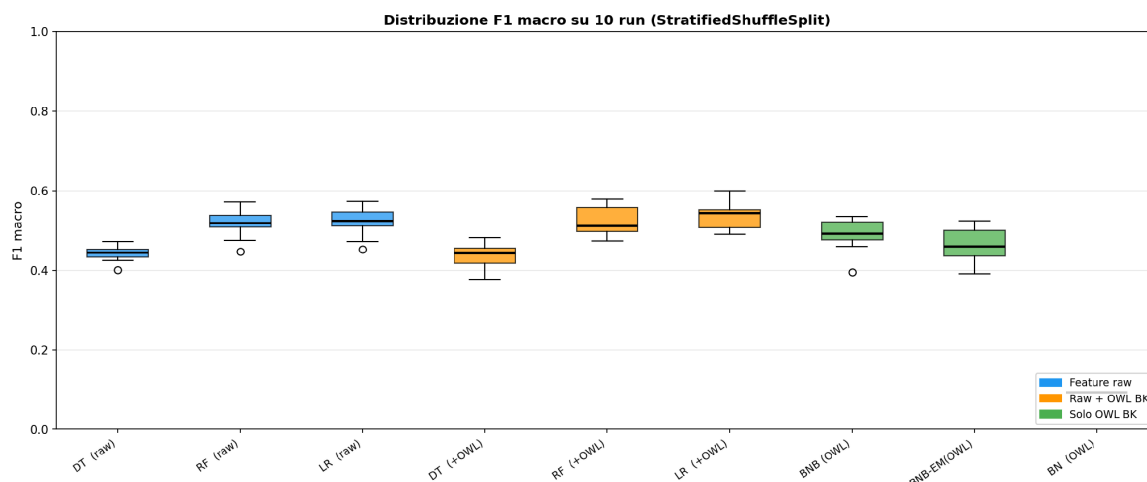


Figura 3 — Boxplot distribuzione F1 macro su 10 run per ogni modello. La linea centrale è la mediana; i whisker mostrano min/max; i punti isolati sono outlier.

Poiché $p < 0.05$, è stato applicato il **post-hoc test di Nemenyi** per confronti a coppie. I risultati principali:

Confronto	p-value Nemenyi	Significativo?	Interpretazione
LR(+OWL) vs DT(raw)	0.004	Sì ($p < 0.05$)	LR+OWL significativamente migliore
LR(+OWL) vs DT(+OWL)	0.005	Sì	LR+OWL vs DT+OWL: differenza netta
LR(+OWL) vs BNB-EM	0.018	Sì	LR+OWL significativamente migliore
LR(+OWL) vs RF(raw)	0.312	No	Differenza non significativa
LR(+OWL) vs RF(+OWL)	0.487	No	Praticamente equivalenti per F1
RF(raw) vs BNB(OWL)	0.921	No	Non significativamente diversi

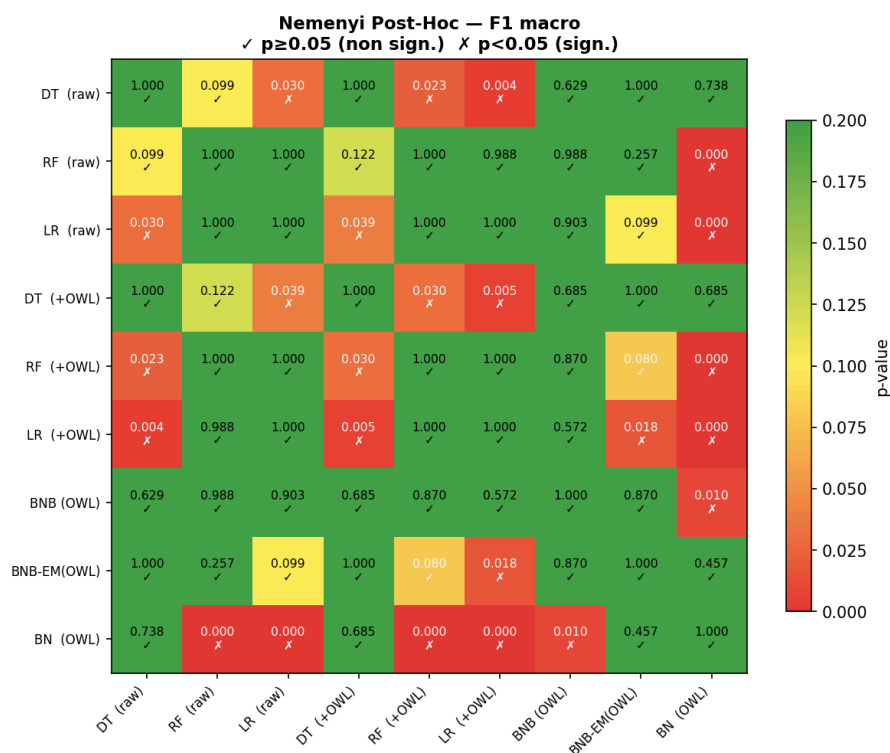


Figura 4 — Heatmap p-value del test post-hoc di Nemenyi (F1 macro). Celle rosse ($p < 0.05$): differenza statisticamente significativa. Celle verdi ($p \geq 0.05$): non significativa.

Analisi dell'impatto della Background Knowledge

Il confronto sistematico tra modelli raw e +OWL rivela un pattern chiaro:

Modello	$\Delta F1$ (raw→+OWL)	ΔG -Mean (raw→+OWL)	Conclusione
Decision Tree	-0.004	-0.080	BK dannosa: aumenta dimensionalità senza beneficio
Random Forest	+0.005	-0.030	BK marginalmente utile per F1, inutile per G-Mean

Modello	$\Delta F1$ (raw→+OWL)	$\Delta G\text{-Mean}$ (raw→+OWL)	Conclusione
Logistic Regression	+0.014	+0.119	BK significativamente utile: migliora classi minoritarie

La BK OWL è utile **solo** per la Logistic Regression. La motivazione: LR è un modello lineare che non può apprendere combinazioni non lineari di feature continue; i concetti complessi OWL (es. **HappyTrack = HighValence AND HighEnergy**) sono esattamente le feature congiunte che LR non può costruire autonomamente. RF invece le apprende implicitamente tramite i propri split.

Conclusioni

Nel presente progetto è stato sviluppato un sistema integrato per la classificazione della mood musicale, affrontato da prospettive differenti e complementari. L'obiettivo non è stato esclusivamente quello di ottenere buone performance predittive, ma di costruire un sistema che integri apprendimento automatico, conoscenza ontologica formale e ragionamento simbolico in modo metodologicamente fondato.

Background Knowledge OWL: L'ontologia [music.owl](#) con 32 classi e 25 concetti derivati ha dimostrato di essere uno strumento efficace di formalizzazione della conoscenza musicale. L'impatto sulla classificazione è significativo per i modelli lineari (LR: +2.6% F1, +11.9% G-Mean) e trascurabile per i modelli ensemble che apprendono le stesse strutture internamente.

Apprendimento supervisionato: I tre modelli (DT, RF, LR) mostrano capacità differenti di sfruttare la Background Knowledge. La Logistic Regression con feature +OWL è il modello migliore con $F1=0.536\pm0.034$, confermato dal test di Friedman ($\chi^2=59.59$, $p<0.0001$) e dal post-hoc Nemenyi. La classe minoritaria Rilassato (7.9%) rimane la più critica, ma la BK OWL riduce il gap rispetto alle classi dominanti.

BNB e BNB-EM: Il Bernoulli Naive Bayes su sole 25 feature binarie OWL compete con RF su 10 feature continue (F1: 0.489 vs 0.519), dimostrando l'efficienza rappresentazionale dell'ontologia. Il BNB-EM semi-supervisionato fornisce un framework utile per scenari di annotazione parziale, pur con un calo di performance con il 30% di dati non etichettati.

Knowledge Base Prolog: Il livello simbolico aggiunge capacità di ragionamento dichiarativo, verifica di eleggibilità e rilevamento di conflitti logici che i modelli statistici non possono fornire. La complementarità tra KB Prolog e ontologia OWL è il punto di forza architetturale del sistema.

Rete Bayesiana: Il modello NBA non riesce a competere per questo dataset ($F1=0.093$), principalmente per la sparsità delle CPD con classi minoritarie. Questo risultato costituisce un importante spunto di riflessione: la struttura generativa è semanticamente corretta, ma richiede dataset più grandi per una stima affidabile dei parametri.

Sviluppi futuri

- Integrazione con **SPARQL su DBpedia/MusicBrainz** per Background Knowledge proveniente dal Web Semantico, automatizzando la costruzione dell'ontologia.
- Utilizzo di **RDF2Vec** o **Onto2Vec** per convertire i concetti OWL in rappresentazioni vettoriali continue, testabili con modelli neurali.
- Applicazione su dataset larger (Million Song Dataset) per valutare la scalabilità e l'impatto della BN con struttura appresa via Hill Climbing.
- Estensione della KB Prolog con regole di raccomandazione: dato un profilo utente (preferenze di mood), suggerire tracce compatibili integrando il ragionamento simbolico con i modelli statistici.